

PRÁCTICA 1
SISTEMAS OPERATIVOS
MARIA ALEXANDRA JIMÉNEZ SUÁREZ

Para 8!

Primero realice la lógica en bloc de notas

```
//Para el ejercicio de 8!  
  
int numFactorial = 8; //$t0  
int val = 1;   //$t1  
int i;  
for (i = 1; i <= numFactorial; i++) {  
    val = val * i;  
}
```

Luego lo pase al programa MARS

```
8factorial.asm*  
1  .text  
2  .global main  
3  
4  main: li $t0, 8      # numero factorial  
5         li $t1, 1  
6         li $t2, 1  
7  
8  loop: bgt $t2, $t0, end  
9         mul $t1, $t1, $t2  
10        addi $t2, $t2, 1  
11        j loop  
12  
13 end:  
14        move $t8, $t1  
15
```

Al ejecutar

Registers			Coproc 1	Coproc 0
Name	Number	Value		
\$zero	0	0		
\$at	1	1		
\$v0	2	0		
\$v1	3	0		
\$a0	4	0		
\$a1	5	0		
\$a2	6	0		
\$a3	7	0		
\$t0	8	8		
\$t1	9	40320		
\$t2	10	9		
\$t3	11	0		
\$t4	12	0		
\$t5	13	0		
\$t6	14	0		
\$t7	15	0		
\$s0	16	0		
\$s1	17	0		
\$s2	18	0		
\$s3	19	0		
\$s4	20	0		
\$s5	21	0		
\$s6	22	0		
\$s7	23	0		
\$t8	24	40320		
\$t9	25	0		
\$k0	26	0		
\$k1	27	0		
\$gp	28	268468224		
\$sp	29	2147479548		
\$fp	30	0		
\$ra	31	0		
pc		4194340		
hi		0		
lo		40320		

Conceptos aprendidos en el vídeo

1. ¿Qué es MIPS?

MIPS es un tipo de procesador y también el lenguaje ensamblador que usa ese procesador.

Sirve para entender cómo la computadora ejecuta instrucciones internamente, trabajando directamente con:

- registros
- memoria
- instrucciones simples

2. Procesador tipo RISC

Un procesador RISC (*Reduced Instruction Set Computer*) se caracteriza por:

- Tener instrucciones simples y cortas
- Ejecutar la mayoría de las instrucciones en un solo ciclo
- Usar muchos registros
- Acceder a la memoria solo mediante instrucciones de carga y almacenamiento

MIPS es un procesador de tipo RISC.

3. ¿Qué es memoria de datos?

La memoria de datos es el espacio donde el programa almacena y lee información durante su ejecución, como:

- Variables
- Resultados intermedios
- Arreglos

En MIPS, los datos se cargan desde memoria a registros para poder operar con ellos, y luego se pueden guardar nuevamente en memoria.

4. Carga inmediata

Instrucción que recibe un argumento (numero), es decir, es una instrucción que asigna directamente un valor numérico a un registro, sin necesidad de leerlo desde memoria.

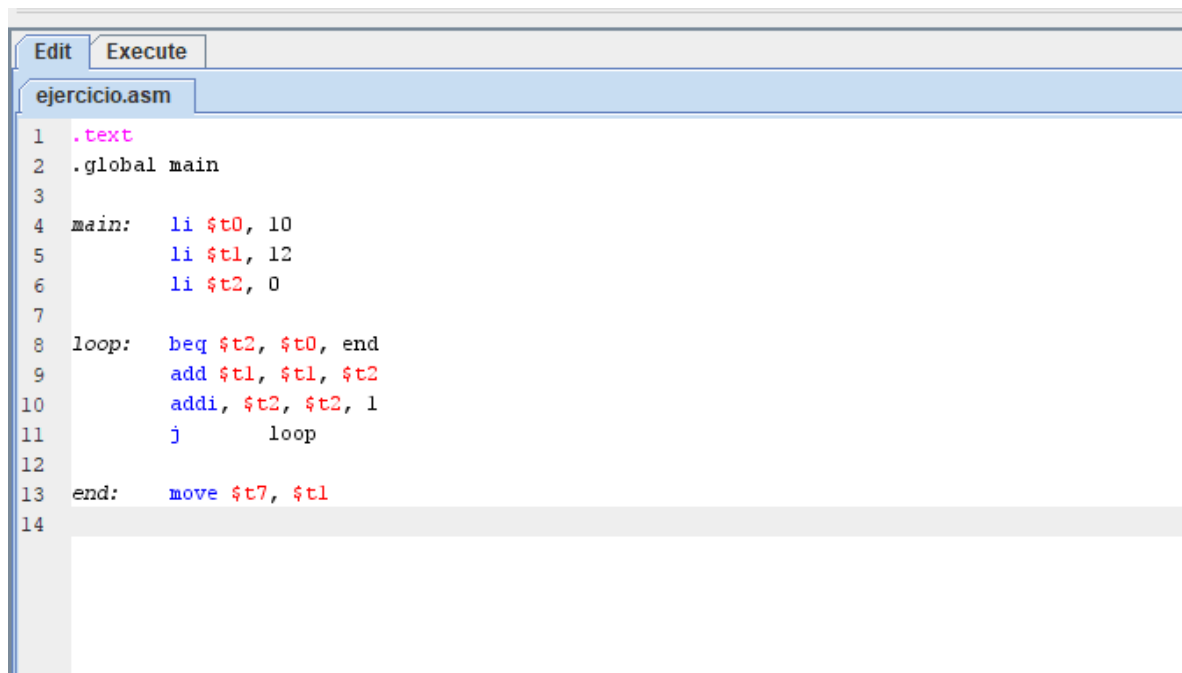
5. Pc : program counter

Instrucción en donde estamos ejecutando actualmente, el PC (Program Counter) es un registro especial que indica la dirección de la instrucción que se está ejecutando actualmente.

- Después de ejecutar una instrucción, el PC avanza a la siguiente
- Puede cambiar su valor con saltos (j, beq, bgt)

Entonces, el PC controla el orden de ejecución del programa.

Primer ejercicio explicado en el vídeo



The image shows a screenshot of a software interface with two tabs: "Edit" and "Execute". The "Edit" tab is active, displaying a file named "ejercicio.asm". The code is written in assembly language and is as follows:

```
1  .text
2  .global main
3
4  main:  li $t0, 10
5         li $t1, 12
6         li $t2, 0
7
8  loop:  beq $t2, $t0, end
9         add $t1, $t1, $t2
10        addi, $t2, $t2, 1
11        j      loop
12
13  end:   move $t7, $t1
14
```

Registers	Coproc 1	Coproc 0
Name	Number	Value
\$zero	0	0
\$at	1	0
\$v0	2	0
\$v1	3	0
\$a0	4	0
\$a1	5	0
\$a2	6	0
\$a3	7	0
\$t0	8	10
\$t1	9	57
\$t2	10	10
\$t3	11	0
\$t4	12	0
\$t5	13	0
\$t6	14	0
\$t7	15	57
\$s0	16	0
\$s1	17	0
\$s2	18	0
\$s3	19	0
\$s4	20	0
\$s5	21	0
\$s6	22	0
\$s7	23	0
\$t8	24	0
\$t9	25	0
\$k0	26	0
\$k1	27	0
\$gp	28	268468224
\$sp	29	2147479548
\$fp	30	0
\$ra	31	0
pc		4194336
hi		0
lo		0